

# Safe Serverless Workflow Orchestration through Typestate Verification, Retry Safety, and Compensation Checking

Anonymous Author(s)

Anonymous Institution  
`anonymous@example.org`

**Abstract.** Serverless workflow orchestration platforms such as AWS Step Functions coordinate fleets of functions and managed services, but their workflows are written as untyped JSON state machines (the Amazon States Language, ASL) whose correctness is checked only at run time. Incompatible task interfaces, out-of-order business transitions, unsafe retries of non-idempotent operations, and missing compensations therefore surface at run time rather than as build errors. We present STEPCHECK, a static verifier that brings four complementary checks—data-contract compatibility, typestate (business-protocol) conformance, retry safety, and Saga compensation completeness—to workflows *before* deployment, and compiles verified workflows back to executable ASL. Because real ASL omits the semantics these checks need, STEPCHECK pairs sound native analyses with a lightweight annotation layer whose defaults are *inferred* from task names and resource bindings, so existing workflows are analysed with zero up-front effort and refined incrementally. Implemented in Rust as a modular frontend/pass pipeline (2.6k LOC), STEPCHECK is evaluated on **193** real Step Functions workflows mined from public AWS repositories: it flags genuine defects in the wild and detects 429 injected defects with 100% recall—where a standard schema validator (`statelint`) catches none of the unsafe retries or missing compensations, STEPCHECK catches all. Name-based inference recovers persistence at 86% but idempotency at only 62% accuracy against an agreement-checked gold set, so semantic findings are surfaced as warnings; verification costs 19  $\mu$ s per workflow; a verified workflow deploys and executes unmodified on AWS Step Functions with zero runtime overhead; and the architecture generalizes to a second workflow language (CNCF Serverless Workflow) with no change to the analyses.

**Keywords:** Serverless computing · Workflow orchestration · AWS Step Functions · Static verification · Typestate · Saga transactions · Domain-specific languages.

## 1 Introduction

Cloud-native applications increasingly decompose business logic into independently deployable functions that are coordinated by managed *workflow orchestration* services. Among these, AWS Step Functions has become one of the most

widely adopted platforms in industry [18,12]. A Step Functions workflow is a state machine written in the *Amazon States Language* (ASL), a JSON configuration format that describes task invocations, branching, parallelism, retries, and error handling, and that executes on top of AWS Lambda and dozens of service integrations [2].

While orchestration platforms simplify *running* distributed workflows, they provide little support for *reasoning about workflow correctness before deployment*. An ASL state machine is an untyped artifact: the platform validates its JSON schema and the existence of transitions, but not whether the data produced by one task matches what the next consumes, whether tasks are invoked in a valid business order, whether an automatically retried task is safe to repeat, or whether a long-running transaction can roll back the effects it has already committed. As a consequence, four recurring classes of defect surface only at run time (detailed in Section 2.2): *contract mismatches* (a task reads a field its predecessor never produces); *protocol/typestate violations* (a task runs out of business order—shipping before payment); *unsafe retries* (a retry re-invokes a non-idempotent operation, duplicating its effect [13]); and *missing compensation* (a persistent operation cannot be rolled back when a later step fails—an incomplete Saga [10]).

A central obstacle to checking these properties is that ASL *omits the very information the checks require*. ASL has no notion of a task’s input or output type, no business-state labels, no idempotency annotation, and no concept of “compensation” beyond a generic `Catch`. Existing language-level approaches sidestep this by introducing a new programming model with a proprietary runtime [6,19,16]; but the millions of workflows already written in ASL cannot benefit, and re-platforming is rarely an option.

*Approach.* We present STEP CHECK, a static verifier that brings four complementary checks to serverless workflows *before deployment* while remaining fully compatible with AWS Step Functions. STEP CHECK is organised around two ideas. First, a *two-layer* analysis model: a set of *native* checks operate soundly on raw ASL with no extra input (structural well-formedness and ASL-semantic data-binding checks), while a set of *semantic* checks (typestate, retry safety, compensation) draw on a lightweight *annotation* layer. Second, because most existing workflows carry no annotations, the missing semantics are *inferred* from task names and resource bindings by a documented, configurable heuristic, giving a zero-effort default analysis that developers can override or refine. Verified workflows are compiled back to executable ASL, so STEP CHECK introduces no runtime component and no lock-in. A typed builder frontend additionally lets greenfield workflows be authored with full static guarantees and the same compilation target.

*Contributions.*

- A practical model and four static analyses —contract/data-flow, typestate, retry safety, compensation completeness— plus a native structural pass (five

- in all), framed operationally rather than through heavy formalism (Sections 3–4).
- A two-layer design that verifies *existing* ASL workflows by pairing sound native checks with an annotation layer whose defaults are inferred from naming/resource conventions (Section 3).
  - STEP CHECK, an open, modular Rust implementation (2.6k LOC) with pluggable frontends (raw ASL, a typed DSL, the CNCF Serverless Workflow DSL), a diagnostics engine, an ASL emitter, and a mutation-based error-injection tool (Section 5).
  - An evaluation on **193 real** workflows: in-the-wild findings, a 429-mutant study (100% recall) contrasted with the `statelint` validator, inference accuracy and warning precision against an agreement-checked gold set, sub-millisecond cost, and a real AWS round-trip (Section 6).

*Research questions.* **RQ1:** Can a static verifier detect the four defect classes in serverless workflows before deployment? **RQ2:** Can the semantics ASL omits be recovered cheaply enough to analyse existing workflows? **RQ3:** What is the cost of verification and what runtime overhead does it impose? We answer these in Section 6.

## 2 Background and Motivation

### 2.1 Step Functions and the Amazon States Language

A Step Functions workflow is a state machine: a `StartAt` entry and a map of named *states*. A `Task` state invokes a Lambda function or a service integration; `Choice` branches; `Parallel` and `Map` fan out; `Wait`, `Pass`, `Succeed`, and `Fail` complete the set. Data flows through a JSON document that each state reshapes with JSONPath fields (`Parameters`, `InputPath`, `ResultPath`, `OutputPath`), where a key suffixed `.$` denotes a path reference. States carry retry (`Retry`) and error-handling (`Catch`) policies. Listing 1.1 shows a representative task drawn verbatim from our corpus.

**Listing 1.1.** A real ASL task state (excerpt, `ship-order`).

```
"Get Customer Status": {
  "Type": "Task", "Resource": "arn:aws:states:::lambda:invoke",
  "Parameters": { "FunctionName": "${LambdaGetCustomerStatus}",
                  "Payload.$": "$.detail.customer" },
  "Retry": [ { "ErrorEquals": ["Lambda.ServiceException"], "MaxAttempts": 6 } ],
  "Next": "Do Fraud Check"
}
```

The platform validates such a definition’s JSON schema and that every transition target exists, then executes it. It does *not* reason about whether `$.detail.customer` is present, whether retrying this task is safe, or how the workflow recovers if a later step fails.

## 2.2 Four classes of latent defect

Our running example is the canonical order-processing workflow `CreateOrder`  $\rightarrow$  `ReserveInventory`  $\rightarrow$  `ChargeCard`  $\rightarrow$  `ShipOrder`, which exhibits all four defect classes when edited carelessly. (1) *Contract*: `ReserveInventory` produces `{orderId, amount}` but `ChargeCard` consumes `total`—a run-time failure on a missing field. (2) *Typestate*: reordering to `ShipOrder`  $\rightarrow$  `ChargeCard` ships before payment (valid JSON, invalid protocol). (3) *Retry*: non-idempotent `ChargeCard` with a broad retry (`States.ALL`) may charge twice. (4) *Compensation*: `ReserveInventory` commits a reservation that leaks if `ChargeCard` fails with no release path. None is caught by the platform; all four are recognised, under-tooled failure modes in cloud-native systems [13,12]. We claim nothing about their prevalence—curated examples are mostly correct (Section 6.2)—only that they escape static checking entirely today; Section 6 measures detection on both unmodified and injected defects.

## 2.3 The information gap

The obstacle to checking these properties statically is not the analysis but the *input*. Raw ASL encodes control flow and JSONPath data plumbing, but it does not record: the *type* of data a task consumes or produces; the *business state* a task represents; whether a task is *idempotent*; or whether a task is *persistent* and which action *compensates* it. A verifier for existing workflows must therefore either (a) restrict itself to properties decidable from ASL alone, or (b) recover the missing semantics. STEP CHECK does both: it runs sound native checks that need nothing beyond ASL, and it recovers the remaining semantics through a lightweight annotation layer whose defaults are inferred from the strong naming and resource conventions that pervade real workflows (Section 3).

# 3 Approach

## 3.1 A two-layer analysis model

STEP CHECK separates what can be checked from ASL alone from what needs recovered semantics. *Native* analyses (Layer A) are sound and require no annotations: structural well-formedness, and ASL-semantic data-binding checks such as a `waitForTaskToken` task that never passes its task token, misuse of the `$$Map` context, and syntactically invalid JSONPath payloads. *Semantic* analyses (Layer B) —typestate, retry safety, compensation, and typed contract compatibility— consume an *annotation* layer that records the task semantics ASL omits. The annotation layer has two sources, in priority order: explicit annotations (authored by hand or produced by the typed DSL frontend), and *inference* that supplies defaults from naming and resource conventions. Findings derived from inferred facts are reported at *warning* severity, while findings derived from declared facts are *errors*; this single rule keeps confidence visible throughout the tool’s output. We are deliberate about what “verification” means at each layer:

the native checks are *sound* and need no input, whereas the semantic checks are only as strong as the semantics they are given—with declared annotations they are precise errors, and in the zero-annotation default they degrade gracefully to best-effort, inference-driven *warnings* (typestate, having no labels to act on, simply stays silent). STEP CHECK is thus a sound linter that becomes a precise verifier as annotations are added.

### 3.2 Workflow model

We model a workflow as a graph of states with a designated start. Beyond its control-flow successors, a *task* carries four pieces of semantic information. **Contract:** a task  $T : A \rightarrow B$  consumes/produces record types (field sets); adjacent  $T_1 : A \rightarrow B$ ,  $T_2 : C \rightarrow D$  *compose* only if  $B$  provides every field  $C$  requires (natively, without declared types, the layer instead checks that JSON-Path references resolve to data the workflow binds). **Typestate:**  $T$  advances a *business state*  $s_{\text{in}} \rightarrow s_{\text{out}}$ ; a workflow is well-ordered when each task consumes the state its predecessor produced, and an optional *protocol* (e.g. *created*  $\rightarrow$  *reserved*  $\rightarrow$  *paid*  $\rightarrow$  *shipped*) lists permitted transitions. **Idempotency:** repeating an idempotent task causes no extra external effect, so it may be retried freely; a non-idempotent one may not be retried on broad error classes. **Persistence/compensation:** a *persistent* task commits a durable resource (reservation, charge, created record) that a later failure should undo, so it needs a *compensation*—an explicit handler or a **Catch** to a compensating state. This vocabulary is deliberately lightweight—no separate formal calculus—yet exactly what the four checks of Section 4 consume.

### 3.3 The annotation layer

Explicit semantics are supplied either through a *sidecar* file attached to an existing ASL workflow (Listing 1.2) or, for greenfield workflows, through a typed builder DSL that emits both the ASL and its annotations (one `.task(...)` call per state carries its schema, protocol edge, idempotency, persistence, and compensation). The two routes resolve to the same internal representation, so the analyses are agnostic to how a workflow was authored.

**Listing 1.2.** A sidecar annotation (TOML), excerpt.

```
[schemas.OrderReserved]
fields = ["orderId", "amount", "reservationId"]
[tasks.ChargeCard]
idempotent = false
persistent = true
compensation = "RefundPayment"
state_in = "OrderReserved"
state_out = "OrderPaid"
```

### 3.4 Inference of default annotations

Most existing workflows carry no annotations, yet their task names and resource ARNs are highly conventional. STEPCheck exploits this with a documented, configurable heuristic that classifies each task from a lowercased *signal* (its name, its **Resource**, and parameters such as **FunctionName**). The heuristic uses four small keyword sets: *read* verbs (**get**, **list**, **describe**, **query**, **validate**, ...) imply idempotent and non-persistent; *write* verbs (**create**, **charge**, **send**, ...) imply non-idempotent; *resource-acquiring* verbs (**reserve**, **pay**, **provision**, **putItem**, ...) additionally imply persistent; and *notification* verbs/resources (**notify**, **publish**, **:sns:**, ...) are non-idempotent but, being fire-and-forget, never persistent. When no keyword matches, the heuristic *abstains*, leaving the task unclassified rather than guessing. Inference thus yields a zero-effort default whose mistakes (Section 6) are visible (warnings) and correctable (an explicit annotation always wins).

### 3.5 Architecture

The pipeline is a straight line: a *frontend* lowers a surface syntax to a format-agnostic intermediate representation (IR); annotation resolution attaches declared and inferred semantics to the IR; a *pass* pipeline runs the five analyses, each emitting coded diagnostics; and an *emitter* lowers verified workflows back to ASL. The design is modular along two axes: a new input format is a new **Frontend**, and a new check is a new **Pass** (Section 5).

## 4 Static Analyses

STEPCheck runs five independent passes over the IR, each emitting diagnostics with a stable code (Table 1), a severity, the offending state, and a remediation note. All are linear in the number of states and recurse into nested **Map/Parallel** sub-machines.

*Structural validation (SC0xxx)*. A reachability traversal from **StartAt** (following normal *and* error transitions) checks that all transition targets exist, that no state is unreachable, that every non-terminal offers a continuation, that a terminal is reachable, that **Choice** states declare a **Default**, and that **Map/Parallel** states are well-formed—sound checks that catch defects AWS reports only at run time.

*Contract / data-flow (SC1xxx)*. Natively, the pass mines every JSONPath reference and checks ASL-semantic bindings: a **waitForTaskToken** task must pass **\$\$Task.Token** (SC1001); the **\$\$Map** context is valid only inside a **Map** (SC1002); and every payload must be a valid path or **States.\*** intrinsic (SC1003). With declared schemas it adds typed checking (SC1010): following control flow through transparent states, each successor’s required fields must be a subset of the producer’s output—the **amount/total** defect of Section 2.2.

**Table 1.** Diagnostic codes emitted by STEP CHECK. Severity † is *error* when derived from declared facts and *warning* when derived from inference.

| Code         | Pass         | Meaning                                                                                                                                    |
|--------------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| SC0001–0010  | structural   | undefined start/transition target, unreachable state, dead-end, non-exhaustive <b>Choice</b> , empty <b>Map/Parallel</b> , malformed state |
| SC1001–1003  | contract     | <b>waitForTaskToken</b> without a task token, <b>\$.Map</b> misuse, invalid JSONPath payload                                               |
| SC1010       | contract     | typed contract mismatch: successor requires a field the predecessor does not produce                                                       |
| SC2001–2002  | typestate    | out-of-order composition; undeclared business-protocol transition                                                                          |
| SC3001†      | retry        | non-idempotent task retried on a broad error class                                                                                         |
| SC4001–4002† | compensation | persistent task with no compensation/handling; compensation names a missing state                                                          |

*Typestate (SC2xxx).* Using **state\_in/state\_out** labels, the pass checks that adjacent tasks compose (predecessor output state = successor input state, SC2001), catching re-orderings such as ship-before-pay, and that each task’s transition is a permitted protocol edge when one is declared (SC2002). Absent annotations it is a no-op.

*Retry safety (SC3xxx).* A task with a **Retry** policy whose **ErrorEquals** includes a broad class (**States.ALL/States.TaskFailed**) is flagged SC3001 if its idempotency resolves to *non-idempotent*. Retries restricted to transient errors (throttling, **ServiceException**) are not flagged, as they fire before any effect.

*Compensation completeness (SC4xxx).* A *persistent* task needs either a declared compensation or a **Catch** path; one with neither leaves its effect uncompensated (SC4001), and a compensation naming a non-existent state is SC4002.

*Worked example.* STEP CHECK reports no findings on the well-formed order workflow. Reordering it to ship before charging (Section 2.2) yields five errors and no false positives: two SC2001 typestate mismatches (e.g. **ShipOrder** consumes *OrderPaid* but its new predecessor produced *OrderReserved*) and three SC1010 contract mismatches (**paymentId**, **amount**, **reservationId** required but not produced upstream)—complementary checks: typestate explains the ordering fault, the contract check pinpoints the broken data.

## 5 Implementation

STEP CHECK is implemented in Rust as a single command-line tool, **stepcheck** (2,631 lines). We chose Rust deliberately: a standalone analyzer with a *serde*-based parser is what makes it possible to ingest the *existing* ASL JSON found

in real repositories —rather than only workflows authored in a new embedded language— and it yields a single dependency-light binary with predictable, sub-millisecond performance.

### 5.1 A modular frontend/pass pipeline

Extensibility rests on two traits (Listing 1.3): a **Frontend** lowers a surface syntax to the IR, a **Pass** reads only the IR and writes diagnostics. Adding an input format is a new **Frontend**; a check, a new **Pass**. We exercised this directly—the tool ships three frontends (typed DSL, raw ASL, CNCF [7]), the last added *without changing the IR or any pass* (Section 6.8).

**Listing 1.3.** The two extension points.

```
pub trait Frontend { fn load(&self, src: &str) -> Result<Workflow>; }

pub trait Pass {
  fn id(&self) -> &'static str;
  fn run(&self, wf: &Workflow, sink: &mut DiagnosticSink);
}
```

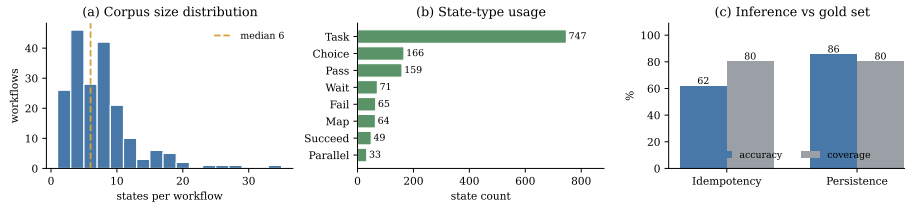
### 5.2 Components

The **workflow IR** is a format-agnostic graph of states (**Task**, **Choice**, **Parallel**, **Map**, **Wait**, **Pass**, **Succeed**, **Fail**) with control flow, JSONPath data-flow fields, retry/catch policies, nested sub-machines, and a slot for resolved annotations. The **Asl frontend** lowers from a generic **serde\_json** value rather than rigid structs—so the diversity of real ASL (intrinsic, **.\$** parameters, the **\$\$** context, **SAM** **{}** placeholders, classic **Iterator** vs. distributed **ItemProcessor**, **JSONata**) never causes a hard failure—and parses all 193 corpus workflows without error. **Annotation resolution** loads a TOML sidecar, resolves schema names to field sets, and fills gaps with the heuristic of Section 3.4. The **passes and diagnostics** implement the five analyses plus an engine that renders findings for a terminal or as JSON. The **Asl emitter** lowers a verified IR back to deployable ASL JSON; it is the DSL frontend’s code-generator and the basis of the AWS round-trip (Section 6)—deploying its output surfaced (and we fixed) a real bug where an explicit **End** was emitted on **Succeed/Fail** states, which Step Functions rejects.

*CLI and error injection.* **stepcheck** exposes **check**, **infer**, **scan**, **stats**, **eval**, **mutate**, and **demo** (the last emits the built-in typed-DSL example to ASL). The **mutate** command injects exactly one defect of a chosen class into a workflow (a broken JSONPath binding, a broad retry on a non-idempotent task, a removed compensation, or a dangling transition) and re-emits deployable ASL; it drives the mutation study of Section 6.

The **2,631** lines divide as: workflow IR 277; three frontends (ASL parse/emit, typed DSL, CNCF) 664; annotations + inference 251; the five analysis passes + manager 650; diagnostics 115; mutation engine 184; and CLI + statistics + evaluation harness 490.





**Fig. 1.** Evaluation overview. (a) Distribution of workflow sizes and (b) state-type usage across the 193-workflow corpus; (c) inference accuracy and coverage for idempotency and persistence against the agreement-checked gold set (Section 6.5).

## 6 Evaluation

We evaluate STEP CHECK on real serverless workflows, asking: can it detect the four defect classes before deployment (RQ1); can it recover the semantics ASL omits cheaply enough to analyse existing workflows (RQ2); and what does verification cost, at build and run time (RQ3)? Every number below is produced by the released tool over a committed corpus and is reproducible with the commands shown.

### 6.1 Setup and corpus (RQ1)

We mined **193** Step Functions workflows from two public AWS repositories (`aws-samples/aws-stepfunctions-examples` and `step-functions-workflows-collection`), taking every file that parses as an ASL state machine. The corpus (Fig. 1) is realistic and diverse: 1,355 states, median 6 per workflow (max 33), and broad use of **Choice** (166), **Map** (64), **Parallel** (33), retries (84 workflows), and catch handlers (51). All experiments ran on a commodity laptop (Rust 1.96 release build).

### 6.2 Findings in the wild (RQ1)

Run unmodified over the corpus (with inference enabled), STEP CHECK flags **86 of 193** workflows. We separate *sound native* findings, which need no annotations and which we manually confirmed, from *inference-driven* findings, which are warnings requiring triage. Among the native findings, nine workflows contain a **Choice** with no **Default** (SC0007)—a recognised cause of **States.NoChoiceMatched** run-time failures—and one is genuinely malformed (SC0010): an `aws-samples` example nests a top-level **QueryLanguage** directive *inside* its **States** block, where Step Functions would reject it. The inference-driven warnings (SC3001: 36; SC4001: 117) mark non-idempotent tasks retried broadly and persistent tasks lacking a compensation path; as Section 6.5 shows, these have measured precision of 72% on gold-covered findings (81% for the compensation warnings) but include false positives (e.g. a **RefundPayment** compensator inferred as needing its own

compensation), which is precisely why they are surfaced as warnings rather than errors.

### 6.3 Mutation-based detection (RQ1)

Because correct real workflows expose few of the deeper defects, we measure detection power by *error injection*. For each defect class we inject exactly one defect per applicable workflow with `stepcheck mutate` and re-verify; a defect counts as *detected* only if the expected diagnostic’s count strictly increases over the unmutated baseline (so we measure detection of the *injected* defect, not pre-existing findings). Across **429** mutants spanning the four classes, STEP CHECK achieves **100% recall** (Table 2, STEP CHECK column), and it never reports the expected defect on the corresponding originals. We stress that this measures *reliability of each check on the defect class it targets*, not detection of arbitrary bugs; the mutations are constructed to fall within each check’s remit. The next subsection contrasts this with what a standard validator detects on the same mutants.

### 6.4 Comparison to a standard validator (RQ1)

The natural baseline is the validation a Step Functions developer already has. `statelint` [3] (the AWS Labs reference linter; we ran v0.8.0) checks an ASL definition against the J2119/ASL schema plus structural rules. We ran it on the same corpus and the same 429 mutants, scoring a mutant *detected* only when statelint emits a new problem that names the injected fault (Table 2). The two tools *agree* on the defects statelint can express: it flags every dangling transition (166/166) and—because a broken JSONPath binding violates the schema—almost every broken contract (140/141). But it detects **0/103** unsafe retries and **0/19** missing compensations: a broad `States.ALL` retry and an absent `Catch` are both schema-valid, so a validator without a notion of idempotency or persistence cannot see them. (A naive count-of-problems metric would appear to “detect” all 103 retry and 18/19 compensation mutants—but only because the injected retry’s float `IntervalSeconds` trips statelint’s integer-vs-float pedantry and a removed `Catch` orphans its handler state; neither complaint is retry-safety or compensation analysis, so we do not credit them.) In the wild the asymmetry is starker still: statelint flags 113/193 workflows with 499 problems, **410 of them** schema-shape nits (e.g. `BackoffRate/IntervalSeconds` number typing) and none touching the four semantic classes, whereas STEP CHECK’s 163 findings are all semantically meaningful. STEP CHECK is thus complementary to and strictly more discriminating than schema validation on the properties that motivate it.

### 6.5 Inference accuracy (RQ2)

The semantic checks depend on recovering idempotency and persistence. We hand-labelled a gold standard of **147 tasks across 23 workflows**—spanning Sagas,

**Table 2.** Injected-defect detection over 429 mutants (delta over baseline): STEPCheck vs. `statelint`. A mutant counts as detected only when the tool emits a new problem naming the injected fault.

| Defect class         | Check  | Applicable | STEPCheck         | <code>statelint</code> |
|----------------------|--------|------------|-------------------|------------------------|
| Dangling transition  | SC0002 | 166        | 166 (100%)        | 166 (100%)             |
| Broken data binding  | SC1003 | 141        | 141 (100%)        | 140 (99%)              |
| Unsafe retry         | SC3001 | 103        | 103 (100%)        | <b>0 (0%)</b>          |
| Missing compensation | SC4001 | 19         | 19 (100%)         | <b>0 (0%)</b>          |
| <b>Total</b>         |        | <b>429</b> | <b>429 (100%)</b> | <b>306 (71%)</b>       |

an inventory domain, account vending, security automation, media/AI pipelines, CQRS, data migration, and more— by each task’s true operational semantics. To guard against labeller bias we labelled the new workflows in *two independent passes* under a written rubric and reconciled the differences; agreement was high (Cohen’s  $\kappa = 1.00$  for idempotency and 0.75 for persistence over the doubly-labelled tasks), and we release the labels for scrutiny. The heuristic abstains on  $\sim 20\%$  of tasks (unconventional verbs such as *confirm*, *audit*, *aggregate*); on the rest it reaches **85.6%** accuracy for persistence but only **61.9%** for idempotency (Table 3). The asymmetry is the headline result of this experiment, and it is honest: persistence (which drives the compensation check) tracks resource-acquiring verbs well, whereas idempotency is genuinely hard from names alone—*cancel/release* compensators and stable-key writes (a DynamoDB `PutItem/UpdateItem` keyed by a primary key) are idempotent *in effect* but read as writes. That high inter-annotator agreement coexists with low heuristic accuracy means the truth is clear but the names do not carry it. This is precisely why inferred findings are *warnings*: a one-line annotation overrides any default, and the residual error is visible, not silent.

*Precision of in-the-wild warnings.* The gold set lets us measure the precision of the inference-driven warnings of Section 6.2 directly: for the 39 SC3001/SC4001 findings that land on a gold-labelled task, a finding is a true positive iff the gold label confirms it. **28/39 (72%)** are true positives—**26/32 (81%)** for the compensation warnings SC4001, but only **2/7** for the broad-retry warnings SC3001, mirroring the weaker idempotency inference that drives them. Every false positive is a compensator (`RefundPayment`, `CancelFlightReservation`) or an idempotent keyed/metadata write—the same patterns above—which is exactly why these are surfaced for triage rather than as errors. RQ2 answer: persistence is recovered well and compensation warnings are mostly actionable; idempotency-driven retry warnings are noisier and benefit most from a declared annotation.

**Table 3.** Inference accuracy vs. the agreement-checked gold set (147 tasks, 23 workflows; accuracy is over non-abstained predictions).

| Property    | Labelled | Predicted | Abstained | Correct | Accuracy | Coverage |
|-------------|----------|-----------|-----------|---------|----------|----------|
| Idempotency | 147      | 118       | 29        | 73      | 61.9%    | 80.3%    |
| Persistence | 147      | 118       | 29        | 101     | 85.6%    | 80.3%    |

### 6.6 Verification cost and runtime overhead (RQ3)

Verification is cheap: the five passes average **18.7**  $\mu$ s/workflow (median 14.9, max 111.8  $\mu$ s); the whole corpus verifies in **3.6 ms**. Because all analysis is ahead-of-deployment and the output is ordinary ASL, the *runtime* overhead is **zero**—no middleware, sidecar, or instrumentation—unlike runtime fault-tolerance and transaction approaches [19,6].

### 6.7 Conciseness and AWS round-trip

The typed DSL is also concise: 20 declarative statements expand to a 121-line ASL definition while additionally encoding five schemas, a four-edge protocol, and per-task idempotency/persistence/compensation that ASL cannot express. To confirm compatibility end-to-end, we deployed the DSL-verified order workflow to real AWS Step Functions *unmodified* (only binding `FunctionName` to a Lambda ARN) as an Express machine: it reached **SUCCEEDED**, the input propagating through all four steps, billed 500 ms / 64 MB; the resources were then torn down. A verified **STEPCHECK** workflow runs on the unmodified industrial platform.

### 6.8 Cross-format generalization: CNCF Serverless Workflow

To test generalization beyond AWS, we added a third frontend for the vendor-neutral CNCF Serverless Workflow DSL [7] and ran it on **66** example workflows from the specification’s repository. The frontend lowers the DSL’s tasks (`call/do/fork/switch/try/...`) to the *same* IR *without changing the IR or any analysis pass*, and parses all 66 with no failures. The structural, typestate, retry-safety, and compensation analyses apply unchanged; only the ASL-specific data-binding checks (SC1001–SC1003) stay silent—correctly, since CNCF uses runtime expressions, not `JSONPath`. A mutation study detects every injected defect with an injection site (retry 4/4, structural 9/9). Crucially, CNCF can declare JSON Schemas, so where it does the typed contract check (SC1010) runs at *full precision with no inference*: a reordered schema-annotated order workflow is rejected with three SC1010 errors from the declared schemas—the same defect inference only approximates on raw ASL, here exact. The asymmetry is informative: CNCF makes data contracts explicit but, like ASL, leaves idempotency and compensation implicit, so those checks still rely on annotation or inference in both ecosystems.

## 6.9 Threats to validity

*Construct.* The mutation study reports 100% recall by design: each mutation targets a defect within a check’s remit, so the result attests to the reliability of each check, not to coverage of all possible bugs. We report it alongside the in-the-wild findings, which are not constructed. *Internal.* Gold labels for idempotency/persistence are judgements; we mitigated bias by labelling in two independent passes under a written rubric with high agreement ( $\kappa = 1.00/0.75$ ), documented the criteria, and release the labels for scrutiny. The contested cases (cancel, refund, keyed writes) are exactly where the paper claims inference is imperfect, and we report the resulting warning precision rather than hide it. *External.* The corpus is drawn from AWS sample repositories, which may over-represent didactic workflows; we mitigate this by taking *all* parseable workflows rather than a curated subset, yielding a realistic spread of sizes and features. *Soundness.* The native contract checks are conservative (they flag only definite faults); typestate and typed-contract precision depend on annotation quality, which is why STEP CHECK reports inferred findings as warnings.

## 7 Related Work

*Serverless orchestration, durable execution, and transactions.* The commercial orchestrators and their trade-offs have been analysed extensively [18,12], and many programming models make orchestration more expressive or data-aware [6]. Durable-execution engines (Temporal, Cadence [16,17]) and systems like Beldi [19] provide exactly-once execution and transactional/Saga semantics *at run time*, via logs and recovery protocols. All target execution—performance, scalability, convenience, fault tolerance. STEP CHECK is complementary: it targets *correctness*, statically, on the most widely deployed platform, adding no runtime machinery; a verified workflow can still run on a durable backend.

*Typestate, sessions, and Saga compensation.* Typestate originates with Strom and Yemini [15] and extends to objects and API protocols [9]; session and multiparty session types give a rich theory of protocol conformance [11]. STEP CHECK adapts the core idea—valid-state-only composition—to workflow tasks, trading full generality for a decidable, linear-time check inside an industrial toolchain. The Saga pattern [10] underpins cloud-native recovery [13], but compensation is usually coded and validated at run time [8]; STEP CHECK adds a *static* completeness check—every persistent operation must have a reachable compensator before shipping.

*Workflow verification and data contracts.* Workflow soundness is verified with Petri nets [1,5] and service compositions with model checking [4]: strong, general guarantees that require dedicated formalisms and are rarely wired into everyday cloud development. Interface compatibility is usually enforced via consumer-driven contracts or statically for REST clients [14]; our contract pass brings this producer/consumer reasoning to the data flow *inside* a workflow. STEP CHECK

deliberately targets a deployed industrial language with lightweight, linear checks and direct compilation. Against representative approaches—AWS Step Functions [2] and CNCF [7] (no static semantic checks), Temporal/Cadence [16,17] and Beldi [19] (runtime retry/Saga, not static), Petri-net/BPEL verification [5,4] (static but heavyweight, non-AWS), and session/typestate types [11,15] (typestate only)—STEPCHECK is, to our knowledge, the first to combine typestate conformance, retry safety, compensation completeness, and data-contract checking in a single *static* verifier that compiles directly to executable AWS Step Functions.

## 8 Conclusion

Serverless workflow orchestration has become a backbone of cloud-native systems, yet its workflows are untyped configuration whose correctness is checked only when they run. We presented STEPCHECK, a static verifier that catches four classes of orchestration defect—contract mismatches, out-of-order business transitions, unsafe retries, and missing compensations—*before* deployment, and compiles verified workflows back to executable Amazon States Language. The key to analysing the workflows that already exist is a two-layer model: sound native checks that need nothing beyond ASL, paired with a lightweight annotation layer whose defaults are inferred from the naming and resource conventions that pervade real workflows.

Our evaluation on 193 real Step Functions workflows supports the three research questions. STEPCHECK detects the targeted defect classes (100% recall over 429 injected defects) where a standard schema validator detects none of the retry or compensation defects, and surfaces genuine issues in unmodified workflows (RQ1); it recovers persistence at 86% and idempotency at 62% accuracy at zero authoring cost against an agreement-checked gold set—enough to make compensation warnings mostly actionable (81% precision) while idempotency-driven retry warnings stay noisier, all surfaced as correctable warnings (RQ2); and it verifies a workflow in 19  $\mu$ s on average while adding zero runtime overhead, with verified workflows running unmodified on real AWS Step Functions (RQ3).

The broader message mirrors the history of type systems: just as static typing moved classes of error from run time to compile time, lightweight static verification can do the same for serverless orchestration, on the platforms developers already use. Future work includes further frontends (SAM, CDK), richer effect/ownership information to sharpen the contract and compensation analyses, and a precise data-flow analysis of `Map/Parallel` item contexts. We release STEPCHECK, the corpus, and the evaluation harness to support replication and reuse.

## References

1. van der Aalst, W.M.P.: The application of Petri nets to workflow management. *Journal of Circuits, Systems and Computers* **8**(1), 21–66 (1998). <https://doi.org/10.1142/S0218126698000043>

2. Amazon Web Services: AWS step functions developer guide (amazon states language). <https://docs.aws.amazon.com/step-functions/> (2026), accessed 2026-06-17
3. Amazon Web Services Labs: statelint: A validator for Amazon states language documents. <https://github.com/aws-labs/statelint> (2024), apache-2.0; v0.8.0. Accessed 2026-06-18
4. Bianculli, D., Ghezzi, C., Spoletini, P.: A model checking approach to verify BPEL4WS workflows. In: IEEE Int. Conf. on Service-Oriented Computing and Applications (SOCA). pp. 13–20 (2007). <https://doi.org/10.1109/SOCA.2007.5>
5. Blondin, M., Mazowiecki, F., Offtermatt, P.: Verifying generalised and structural soundness of workflow nets via relaxations. In: Int. Conf. on Computer Aided Verification (CAV). pp. 435–458 (2022). [https://doi.org/10.1007/978-3-031-13188-2\\_23](https://doi.org/10.1007/978-3-031-13188-2_23)
6. Burckhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C., Meiklejohn, C.S.: Durable functions: Semantics for stateful serverless. Proc. ACM Program. Lang. **5**(OOPSLA) (2021). <https://doi.org/10.1145/3485510>
7. Cloud Native Computing Foundation: Serverless workflow specification. <https://serverlessworkflow.io/> (2026), accessed 2026-06-17
8. Colombo, C., Pace, G.J.: Comprehensive monitor-oriented compensation programming. arXiv preprint (2014), arXiv:1404.0849
9. DeLine, R., Fähndrich, M.: Typestates for objects. In: European Conf. on Object-Oriented Programming (ECOOP). pp. 465–490 (2004). [https://doi.org/10.1007/978-3-540-24851-4\\_21](https://doi.org/10.1007/978-3-540-24851-4_21)
10. Garcia-Molina, H., Salem, K.: Sagas. In: ACM SIGMOD Int. Conf. on Management of Data. pp. 249–259 (1987). <https://doi.org/10.1145/38713.38742>
11. Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. In: ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL). pp. 273–284 (2008). <https://doi.org/10.1145/1328438.1328472>
12. Kulkarni, V., Reddy, N., Khare, T., Prasad, A.S., Babu, C., Simmhan, Y.: Characterizing FaaS workflows on public clouds: The good, the bad and the ugly. arXiv preprint (2025), arXiv:2509.23013
13. Mohammad, M.: Resilient microservices: A systematic review of recovery patterns, strategies, and evaluation frameworks. arXiv preprint (2025), arXiv:2512.16959
14. Seco, J.C., Ferreira, P., Lourenço, H., Ferreira, C., Ferrão, L.: Robust contract evolution in a typesafe microservices architecture. The Art, Science, and Engineering of Programming **4**(3) (2020). <https://doi.org/10.22152/programming-journal.org/2020/4/10>
15. Strom, R.E., Yemini, S.: Typestate: A programming language concept for enhancing software reliability. IEEE Transactions on Software Engineering **SE-12**(1), 157–171 (1986). <https://doi.org/10.1109/TSE.1986.6312929>
16. Temporal Technologies: Temporal: Durable execution and workflow orchestration. <https://temporal.io/> (2026), accessed 2026-06-17
17. Uber Technologies: Cadence: Fault-oblivious stateful workflow orchestration. <https://cadenceworkflow.io/> (2026), accessed 2026-06-17
18. Wen, J., Liu, Y.: An empirical study on serverless workflow service. arXiv preprint (2021), arXiv:2101.03513
19. Zhang, H., Cardoza, A., Chen, P.B., Angel, S., Liu, V.: Fault-tolerant and transactional stateful serverless workflows. In: USENIX Symp. on Operating Systems Design and Implementation (OSDI) (2020), arXiv:2010.06706